

V6.0

Always-on DAI-to-DAI

Architecture firmware SOF · i.MX8MP Debix Model AB

Phase **1a.4** · Étape **E0** (loopback minimal)

Date : **2026-05-08**

Auteur : **Michael Jouannigot**

SOF firmware : `feature/v6-always-on-async` · HEAD `0b28f8074`

Yocto : `feature/audio-platform-v2` · HEAD `73f2b2bf`

Topology : `sof-imx8mp-tac5212-V6.0.m4`

Sommaire

1. Résumé exécutif
2. Problématique produit
3. Limites du SOF standard PCM-driven
4. Architecture cible V6.0
5. Invariants hardware SAI7 + TAC5212
6. Cycle de vie SOF — pourquoi le walk standard ne marche pas
7. Code clé — F4 handler dual-anchor
8. Patches V6.0 (firmware + kernel + topology)
9. Plage mailbox debug
10. Plan de validation E0 → E5
11. Plans de repli (V6.0a, firmware custom)
12. Références

1. Résumé exécutif

V6.0 est une refonte de l'architecture firmware SOF pour le DSP HiFi4 du i.MX8MP visant à **découpler le DMA hardware audio du lifecycle ALSA**. Le DSP exécute en permanence un pipeline always-on qui boucle SAI7 RX → buffer B0 → SAI7 TX, indépendamment de l'ouverture/fermeture des PCM.

Cette architecture résout 4 limitations du design SOF standard :

1. Le BCLK ne s'arrête jamais → TAC5212 PLL toujours lockée → pas de glitch ni de re-lock à chaque close PCM.
2. L'audition continue des micros vers les speakers ne nécessite aucun process userspace.
3. Les PCM ASIO (8 IN + 8 OUT) deviennent des piggyback sur le pipeline DSP-only — leur ouverture/fermeture ne touche pas le DMA hardware.
4. Le NPU tap (export samples post-effets vers DDR/NPU) reste actif en permanence, plus dépendant de l'état d'un PCM ouvert.

L'architecture introduit **3 modifications structurelles dans SOF IPC3** :

- **Flag pipeline always-on** qui survit à STOP/PAUSE.
- **Pipeline NO_HOST** avec deux DAI endpoints (au lieu d'un host PCM + un DAI).
- **Handler IPC dual-anchor** qui trigger les deux DAIs séparément (RX d'abord, puis TX), car le walk topologique unique ne traverse pas la chaîne RX → B0 → TX.

État actuel : 5/6 patches firmware (F1-F5) commités · 5/5 patches kernel (K0-K5) commités · 4/6 patches topology (T1-T6) commités · Étape E0 (loopback minimal) en attente de validation hardware sur board.

2. Problématique produit

2.1. Contexte fonctionnel

Le produit cible est un studio audio embarqué (Debix Model AB) qui doit exposer simultanément :

- Une **audition continue 24/7** des entrées micros (TAC5212 line-in 8 voies TDM) vers les sorties speakers (TAC5212 line-out 8 voies TDM), traitée par des effets DSP (8 strips IN + matrix 16×8 + 8 strips OUT).
- Deux PCMs ALSA **ASIO 8ch s32le 48 kHz** :
 - ASIO IN : capture du flux post-effets-IN vers Linux (pour traitement userspace, enregistrement, NPU).
 - ASIO OUT : injection de Linux vers la matrix 16×8 src0 (mix avec les micros traités).
- Un **NPU tap** qui hook `dai_dma_cb` côté SAI TX et copie les samples post-effets vers une zone reserved-memory accessible au NPU.
- Une **GUI temps réel** (E8) avec FFT du tap NPU + FFT d'une voie sélectionnée + réglage live des effets DSP/TAC.

2.2. Contraintes hardware

Composant	Caractéristique
SoC	NXP i.MX8MP — Cortex-A53 + HiFi4 DSP + NPU + Vivante GPU
SAI7	TX master (génère BCLK/FSYNC), RX slave (boucle PCB hardware)
TAC5212	Codec 4 ADC + 4 DAC, TDM 8 slots × 32 bit, PLL sur BCLK
Format audio	8 canaux S32 LE @ 48 kHz, period DMA 2 ms (NON-NÉGOCIABLE)
SDMA	i.MX SDMA (pas EDMA) — scripts SHP2MCU/MCU2SHP, watermark 64 mots

3. Limites du SOF standard PCM-driven

SOF IPC3 suppose une topologie **host-centric** : chaque pipeline a un host PCM à une extrémité et un DAI à l'autre. Le lifecycle audio est piloté par les transitions ALSA `OPEN → HW_PARAMS → PREPARE → START → STOP → CLOSE`, qui propagent au DAI via `pipeline_trigger()`.

Conséquence pour notre besoin :

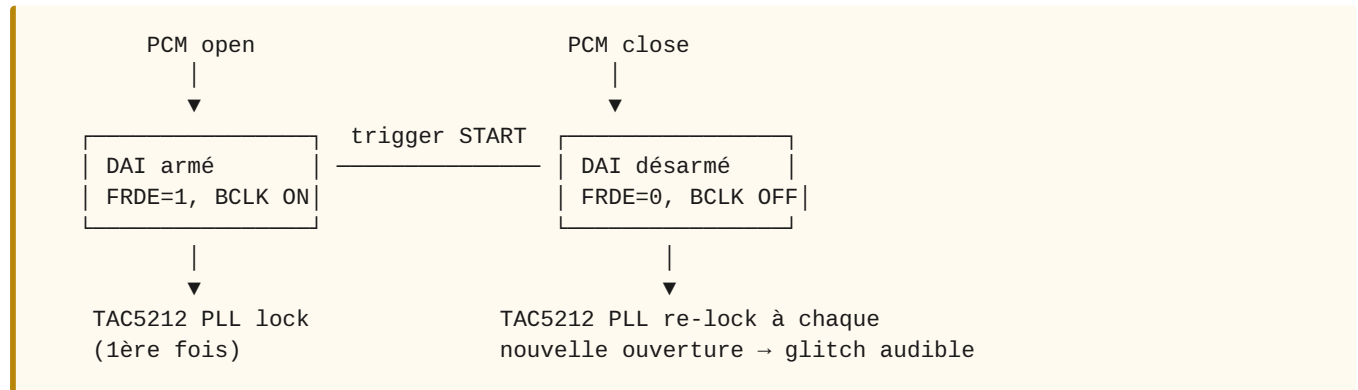


Fig. 1 — Cycle SOF standard : BCLK suit le lifecycle PCM, TAC5212 PLL re-lock à chaque cycle.

Problème bloquant : le couplage PCM lifecycle ↔ DMA hardware empêche l'audition continue (qui demande BCLK 24/7) et fait glitcher chaque ouverture/fermeture de PCM ASIO. Validé empiriquement en E5/E6.b par 8 itérations matrix_2x8 et 6 investigations critic — la cause est **structurelle**.

4. Architecture cible V6.0

4.1. Vue d'ensemble

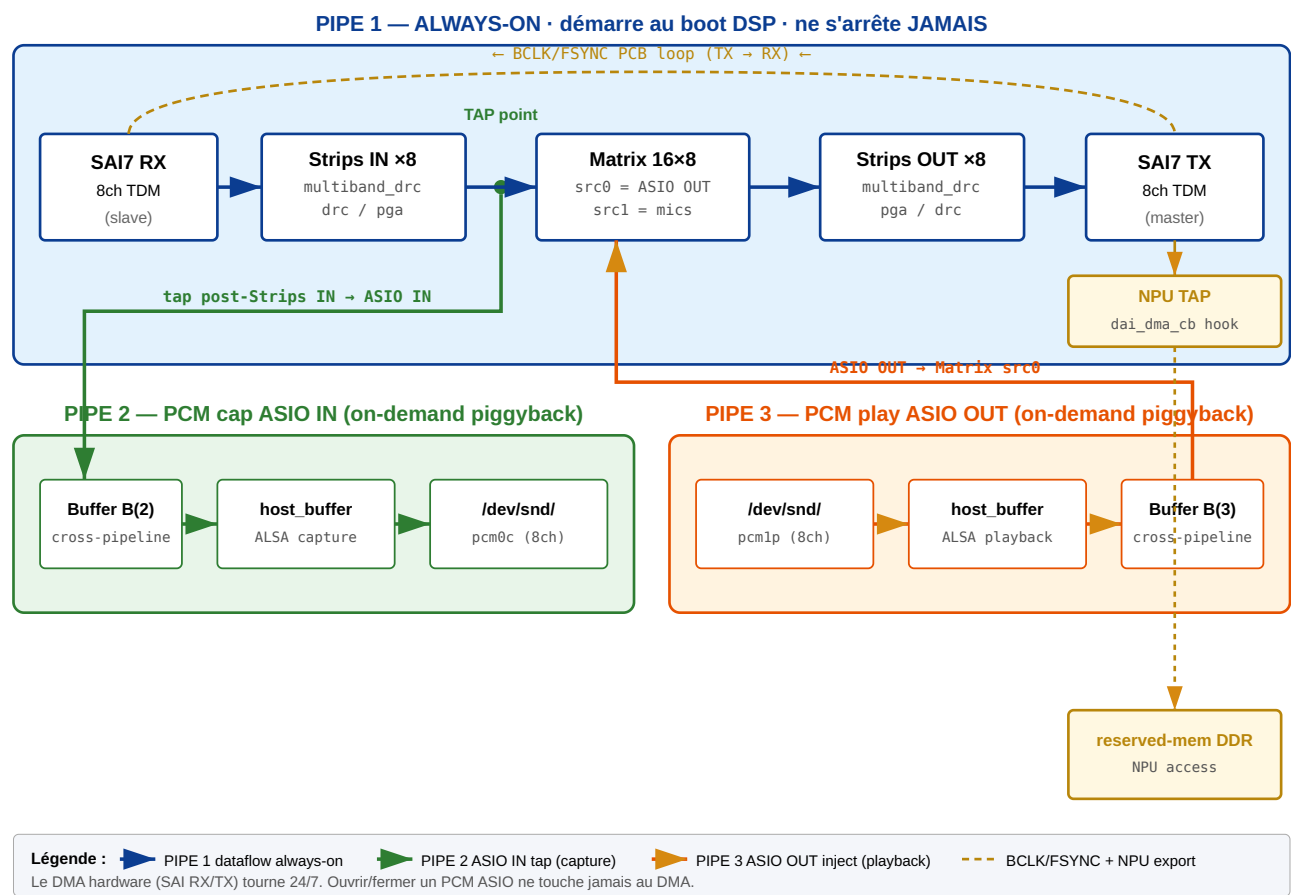


Fig. 2 — Architecture V6.0 complète. PIPE 1 always-on (bleu) ; PIPE 2 ASIO IN (vert) tap intra-PIPE 1 entre Strips IN et Matrix ; PIPE 3 ASIO OUT (orange) inject sur Matrix src0 ; NPU TAP (jaune) hooke `dai_dma_cb` SAI TX vers reserved-memory.

4.2. Garanties non-négociables

Garantie	Comment
DMA SAI RX/TX always-on dès boot DSP	K1 trigger PRE_START + START au probe, F3 swallow STOP/PAUSE
BCLK/FSYNC continus 24/7	sai_set_config() arme TERE+TRCE+MCLK_EN dès DAI_CONFIG, FCONT=1, idempotence patch
Period DMA 2 ms	SCHEDULE_TIME_DOMAIN_DMA, jamais TIMER, jamais 1 ms
Ouverture/fermeture PCM ASIO transparente	K4 skip STOP IPC vers DAI quand pipeline always-on
NPU tap V3.2.2 préservé	hook <code>dai_dma_cb</code> SAI TX vers reserved-memory dédiée

5. Invariants hardware SAI7 + TAC5212

5.1. Topologie clock

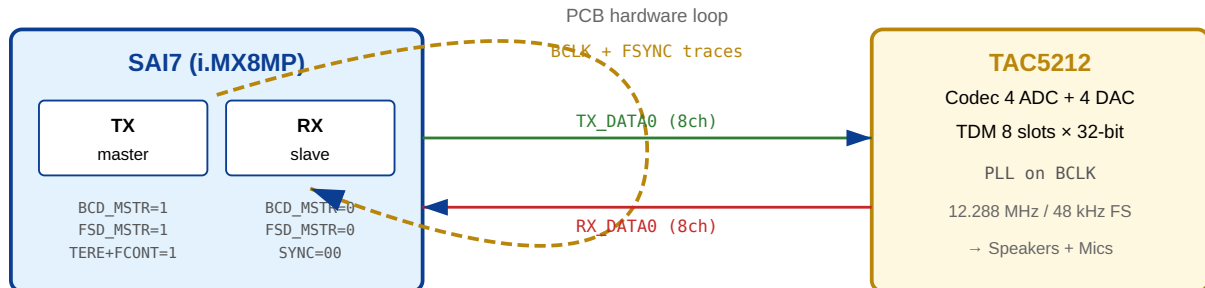


Fig. 3 — Topologie clock SAI7 + TAC5212. TX master génère BCLK/FSYNC, bouclé en PCB vers RX slave et vers TAC5212.

5.2. Découplage clock vs dataflow (patch idempotence)

Le driver SAI a déjà un mécanisme qui sépare l'armement du clock (TERE/TRCE/MCLK_EN) du flag DMA request (FRDE), via le flag `sai->configured` :

```
/* sof/src/drivers/imx/sai.c - sai_start TX, branche idempotence */
if (direction == DAI_DIR_PLAYBACK && sai->configured) {
    /* Clock already running since sai_set_config; only flip FRDE
     * to enable the DMA request. TERE/TRCE/MCLK stay set 24/7
     * to keep the codec PLL locked. */
    sai_update_bits(dai, REG_SAI_TCSR, REG_SAI_CSR_FRDE, REG_SAI_CSR_FRDE);
    return 0;
}
```

Conséquence : pour V6.0, l'invariant "TX clock first" est **satisfait par construction**. À aucun moment du lifecycle, le BCLK ne s'interrompt. Cela élimine la phase 1 du séquençement asymétrique TX_clock → RX_DMA → TX_DMA — il reste juste à ordonner RX trigger avant TX trigger dans le handler F4.

6. Cycle de vie SOF — pourquoi le walk standard ne marche pas

6.1. Le walk topologique

SOF utilise un walk topologique pour propager les commandes (params, prepare, trigger) à travers les composants d'un pipeline. `pipeline_for_each_comp(comp, ctx, dir)` traverse les buffers attachés à `comp` selon `dir` :

direction	valeur	liste de buffers traversée
PPL_DIR_DOWNSTREAM	0 (= SOF_IPC_STREAM_PLAYBACK)	<code>comp->bsink_list</code> (vers les sinks)
PPL_DIR_UPSTREAM	1 (= SOF_IPC_STREAM_CAPTURE)	<code>comp->bsource_list</code> (vers les sources)

6.2. Pour un PCM standard (1 host + 1 DAI)

Topology playback : `HOST → comp1 → comp2 → DAI(TX)`

```
pipeline_trigger_run(host, dir = host->direction = PLAYBACK = 0 = DOWNSTREAM)
  walk DOWNSTREAM depuis HOST → suit bsink_list → atteint comp1, comp2, DAI ✓
```

Topology capture : `DAI(RX) → comp1 → comp2 → HOST`

```
pipeline_trigger_run(host, dir = host->direction = CAPTURE = 1 = UPSTREAM)
  walk UPSTREAM depuis HOST → suit bsource_list → atteint comp2, comp1, DAI ✓
```

6.3. Pour V6.0 NO_HOST DAI-to-DAI : LE WALK STANDARD ÉCHOUE

Topology V6.0 : `DAI(RX) → B0 → DAI(TX)`
 source_comp sink_comp

Tentative 1 : `anchor = source_comp = DAI_RX, dir = CAPTURE = UPSTREAM`

```
  walk UPSTREAM depuis DAI_RX → suit bsource_list[DAI_RX]
  → bsource_list[DAI_RX] = [] (DAI source = début du pipeline)
  → walk s'arrête immédiatement après DAI_RX x
  → B0 et DAI_TX jamais visités
```

Tentative 2 : `anchor = sink_comp = DAI_TX, dir = PLAYBACK = DOWNSTREAM`

```
  walk DOWNSTREAM depuis DAI_TX → suit bsink_list[DAI_TX]
  → bsink_list[DAI_TX] = [] (DAI sink = fin du pipeline)
  → walk s'arrête immédiatement après DAI_TX x
  → B0 et DAI_RX jamais visités
```

Verdict : aucun walk topologique unique ne couvre la chaîne complète `RX → B0 → TX` dans V6.0 NO_HOST. Le mécanisme SOF standard est conçu pour des topologies host-centric, pas DAI-to-DAI. Investigation critic [5f4ee23e](#) et [f25615e1](#) : 5/6 workers convergent indépendamment sur ce point.

6.4. Solution : dual-anchor RX-then-TX

Le handler F4 fait **deux walks séparés**, depuis chaque endpoint, dans l'ordre RX → TX :

Phase PRE_START :

```
pipeline_params(p, source_comp, params_RX_capture) ← config RX seul
pipeline_prepare(p, source_comp)                  ← prepare RX seul
pipeline_params(p, sink_comp, params_TX_playback) ← config TX seul
pipeline_prepare(p, sink_comp)                     ← prepare TX seul
```

Phase START :

```
pipeline_trigger_run(p, source_comp, START) ← trigger RX (slave)
pipeline_trigger_run(p, sink_comp, START)  ← trigger TX (master)
```

Avantages :

- bsource_list[DAI_RX] vide → walk RX ne touche QUE DAI_RX (pas de double-config B0)
- bsink_list[DAI_TX] vide → walk TX ne touche QUE DAI_TX
- B0 partagé : configuré comme dd->local_buffer dans chaque dai_common_params()
- Ordre RX-then-TX : RX commence à remplir B0 avant que TX ne consomme
- TX clock déjà active depuis sai_set_config → pas d'attente nécessaire entre RX et TX

7. Code clé — F4 handler dual-anchor

Fichier : `sof/src/ipc/ipc3/handler.c` · fonction `ipc_glb_tplg_pipe_trigger` · lignes 1406–1593.

```

static int ipc_glb_tplg_pipe_trigger(uint32_t header)
{
    struct sof_ipc_pipe_trigger msg;
    struct pipeline *p;
    struct comp_dev *src, *snk;
    int ret;

    IPC_COPY_CMD(msg, ipc->comp_data);

    /* Récupère la pipeline ciblée par le kernel via pipeline_id */
    ipc_pipe = ipc_get_pipeline_by_id(ipc, msg.pipeline_id);
    if (!ipc_pipe || !ipc_pipe->pipeline) return -EINVAL;
    p = ipc_pipe->pipeline;

    /* V6.0: applique les attributs always-on / ignore-stop / no-host
     * à la pipeline cible (IPC3 ne propage pas ces attributs depuis la
     * topology, contrairement à IPC4). */
    p->attributes |= PIPELINE_ATTR_ALWAYS_ON
                  | PIPELINE_ATTR_IGNORE_STOP
                  | PIPELINE_ATTR_NO_HOST;

    src = p->source_comp; /* DAI capture (RX, slave) */
    snk = p->sink_comp; /* DAI playback (TX, master) */

    if (msg.cmd == COMP_TRIGGER_PRE_START) {
        /* Phase 1 : params + prepare RX */
        struct sof_ipc_pcm_params params_src = build_params(
            src->ipc_config.id, SOF_IPC_STREAM_CAPTURE,
            msg.rate, msg.channels, msg.frame_fmt);
        ret = pipeline_params(p, src, &params_src);
        if (ret < 0) return ret;
        ret = pipeline_prepare(p, src);
        if (ret < 0) return ret;

        /* Phase 2 : params + prepare TX */
        struct sof_ipc_pcm_params params_snk = build_params(
            snk->ipc_config.id, SOF_IPC_STREAM_PLAYBACK,
            msg.rate, msg.channels, msg.frame_fmt);
        ret = pipeline_params(p, snk, &params_snk);
        if (ret < 0) return ret;
        ret = pipeline_prepare(p, snk);
        if (ret < 0) return ret;
    }

    /* Trigger order: RX (slave) FIRST, then TX (master).
     * Hardware: TX clock is continuous (sai_set_config IDEMPOTENCE
     * keeps TERE+TRCE permanently set; BCLK/FSYNC always running for
     * codec PLL stability). TX BCLK/FSYNC are physically wired to RX
     * clock inputs. So enabling RX (FRDE+dma_start) needs no clocks-up
     * wait – clocks are there. Then TX must start AFTER B0 has data
     * from RX, otherwise TX FIFO underflows and SAI hardware
     * auto-disables TE/BCE/FRDE on FEF assertion. */
    ret = pipeline_trigger_run(p, src, msg.cmd);
    if (ret < 0) return ret;
    ret = pipeline_trigger_run(p, snk, msg.cmd);
    if (ret < 0) return ret;

    /* Diag mailbox : TCSR snapshots aux offsets 0x7B0/4/8/C, compteurs 0x7C0/4 */
    diag_tcsr_snapshot(msg.cmd);
    return ret;
}

```

8. Patches V6.0

8.1. Firmware DSP (branche feature/v6-always-on-async)

ID	Fichier	Rôle	Statut
F1	src/include/sof/audio/pipeline.h	Flags PIPELINE_ATTR_ALWAYS_ON / IGNORE_STOP / NO_HOST	✓ commité
F2	handler.c:ipc_glb_tplg_pipe_complete	Auto-trigger après tplg load (via cmd 0x014)	✓ commité
F3	pipeline-stream.c:632-644	Swallow STOP/PAUSE pour IGNORE_STOP	✓ commité
F4	handler.c:ipc_glb_tplg_pipe_trigger	Dual-anchor RX-then-TX (commit 0b28f8074)	✓ commité
F5	module_adapter_ipc3.c:142-185	Short-circuit module_sources_all_intra_pipeline	✓ commité
F6	matrix_2x8.c	Silence-on-empty (output-driven garanti)	⏸ V6.1

8.2. Kernel ASoC (patcher Yocto additif)

Fichier : meta-local/recipes-kernel/linux/files/apply-v6-always-on.py

ID	Fichier kernel	Rôle
K0	tokens.h + header.h + topology.h	Token 224, cmd IPC 0x014, struct étendue rate/ch/fmt/dir
K2	ipc3-topology.c	Parser le token SOF_TKN_PIPE_ALWAYS_ON
K4	sof-audio.h	Champ bool always_on dans snd_sof_widget
K5	ipc3-topology.c	Helper sof_ipc3_send_pipe_trigger
K1	ipc3-topology.c	Trigger PRE_START + START après set_up_all_pipelines

8.3. Topology m4

ID	Fichier	Rôle	Statut
T1	tokens.m4	Token SOF_TKN_PIPE_ALWAYS_ON	✓
T2	pipeline.m4	Macro PIPELINE_ALWAYS_ON_ADD	✓
T3	(intégré T6)	Pipeline DAI-to-DAI inline	✓
T6	sof-imx8mp-tac5212-V6.0.m4	Topology E0 complète	✓
T4	pipe-host-only-capture.m4	PIPE 2 ASIO IN piggyback	⏸ V6.1
T5	pipe-host-only-playback.m4	PIPE 3 ASIO OUT piggyback	⏸ V6.1

9. Plage mailbox debug

SRAM_DEBUG_SIZE = 2048 B (0x000–0x7FF). Carte exhaustive dans `TESTS/MAILBOX_DEBUG_MAP.md`.
Plage propre confirmée libre : **0x470–0x4FF (144 B)**.

9.1. Adresses utilisées par V6.0 actuellement

Offset	Sens	Source
0x720	sai_start TX count	sai.c (existant E6.b)
0x728	TCSR snapshot inside sai_start TX	sai.c
0x730	sai_start RX count	sai.c
0x7B0	TCSR après PRE_START handler exit	handler.c F4
0x7B4	TCSR après START handler exit	handler.c F4
0x7B8	TCSR après STOP handler exit	handler.c F4
0x7BC	TCSR après PAUSE handler exit	handler.c F4
0x7C0	PRE_START handler exit count	handler.c F4
0x7C4	START handler exit count	handler.c F4

9.2. Plage réservée pour nouveau diag E0/E1

Offset	Sens proposé
0x470	RCSR snapshot après START handler exit
0x474	src trigger return code (RX)
0x478	snk trigger return code (TX)
0x47C	src pipeline_trigger_run call count
0x480	snk pipeline_trigger_run call count
0x484	RCSR snapshot après PRE_START
0x488–0x4FF	réserve V6.1

Règle stricte : aucune adresse mailbox ne doit être réutilisée d'un commit à l'autre dans la même session de diag. Cause documentée : collisions historiques B (0x644) et D (0x740) dans `MAILBOX_DEBUG_MAP.md` ont produit des analyses erronées.

10. Plan de validation E0 → E5

Étape	Patches	Test cible	Critère
E0	F1+F2+F3+F4+F5 + K0-K5 + T1+T2+T3+T6	Boot DSP + loopback hardware mics → speakers sans aucun PCM ouvert	BCLK 12.288 MHz continu sur scope ; TCSR final TE=1, FRDE=1, FEF=0 ; audio loopback audible
E1	+ T4	Ajout PCM cap ASIO IN	arecord capture 8ch sans casser le loopback E0
E2	+ T5 + K4 amélioré	Ajout PCM play ASIO OUT	aplay injecte dans la matrix sans casser le loopback E0
E3	+ matrix_2x8 + strips IN/OUT (V6.1)	Effets DSP en temps réel	amixer cset modifie gains sans glitch
E4	+ NPU tap (hook dai_dma_cb)	Export samples post-effets	NPU tap V3.2.2 capture continue dans reserved-memory
E5	—	Full duplex final + GUI E8	aplay+arecord simultané, FFT live, réglage effets DSP

Chaque étape doit produire une fiche `TESTS/TESTS_V6.0_E<n>.md` avec firmware md5, topology md5, résultats détaillés, statut GO/REVIEW/NOK.

11. Plans de repli

11.1. Plan B — V6.0a (E6.a+) sans always-on

Si E0 échoue empiriquement (pas de BCLK même avec dual-anchor + always-on), bascule vers architecture E6.a + chaîne d'effets en sortie :

```
PIPE 1 cap : SAI RX → eq×8 + drc×8 + pga×8 → host PCM 0 (ASIO IN, 8ch)
PIPE 2 play: host PCM 1 (ASIO OUT) → deinterleave_8 → mixer16 → interleave_8
                                     → eq×8 + drc×8 + pga×8 → SAI TX
```

Limitation : pas de loopback hardware mics → TX. Pas de matrix mics tap. À étudier mécanisme alternatif pour audition continue.

11.2. Plan C — Firmware DSP custom sans SOF

Si V6.0 et V6.0a échouent : abandonner SOF complètement.

- Reprendre uniquement les modules d'effets (eq_iir.c, drc.c, pga.c, multiband_drc.c) comme bibliothèques C portables.
- Firmware Zephyr-only minimal sur le M7.
- Drivers SAI7 + SDMA + loopback simple direct.
- Driver custom Linux pour exposer 2 PCMs ALSA via mailbox/IPC custom.

Validé par le client comme option viable (produit commercial, pas Arduino).

12. Références

12.1. Documentation projet

- TESTS/PLAN_V6.0_ALWAYS_ON.md — Plan opérationnel détaillé
- TESTS/PATCHES_V6.0_DETAILS.md — Détails code de chaque patch
- TESTS/MAILBOX_DEBUG_MAP.md — Carte exhaustive mailbox + collisions historiques
- TESTS/TESTS_V6.0_E0.md — Fiche test E0 baseline pré-board
- ARCHI/ARCHI_V6.0_always_on_dai_to_dai.md — Doc archi V6.0 (source markdown de ce PDF)

12.2. Investigations critic clés

- Job 5f4ee23e — Walk direction wrong (5/6 workers convergent, 988s)
- Job f25615e1 — Invariant TX-master/RX-slave (5/6 workers convergent)
- Jobs E6.b historiques : 2566539a , 183689a7 (test isolation B5), 37ee64a1

12.3. Mémoires bloquantes

- feedback_dma_2ms_definitive — period 2ms NON-NÉGOCIABLE
- sof_sai_idempotence_patch — TX clock continue, ne PAS revert
- feedback_unique_mailbox_addresses — adresses mailbox uniques par diag
- feedback_critic_only_committed — critic uniquement sur code pushé
- feedback_post_compact_checklist — 7 points post-compaction
- feedback_test_fiches — format fiches TESTS
- feedback_architecture_doc_per_phase — ce doc en est l'application

12.4. Commits clés

Repo	Branche	HEAD
SOF	feature/v6-always-on-async	0b28f8074 — F4 dual-anchor RX-then-TX
yocto-nxp-debix	feature/audio-platform-v2	73f2b2bf — TESTS/ + ARCHI/ persistés